UT3 · Formularios, validación y objetos

GET, POST, errores y operaciones de biblioteca

Desarrollo web en entorno servidor · 2.º DAW

Juan Tamboleo López-Brea
Departamento de Informática
IES Ramón Arcas Meca · Lorca
Curso 2026/2027

Índice de la presentación

03–14

Recibir datos

Peticiones y controles HTML.

15–31

Validar y responder

Reglas, errores y un formulario completo.

32–46

Organizar la validación

De las reglas al servicio y al modelo.

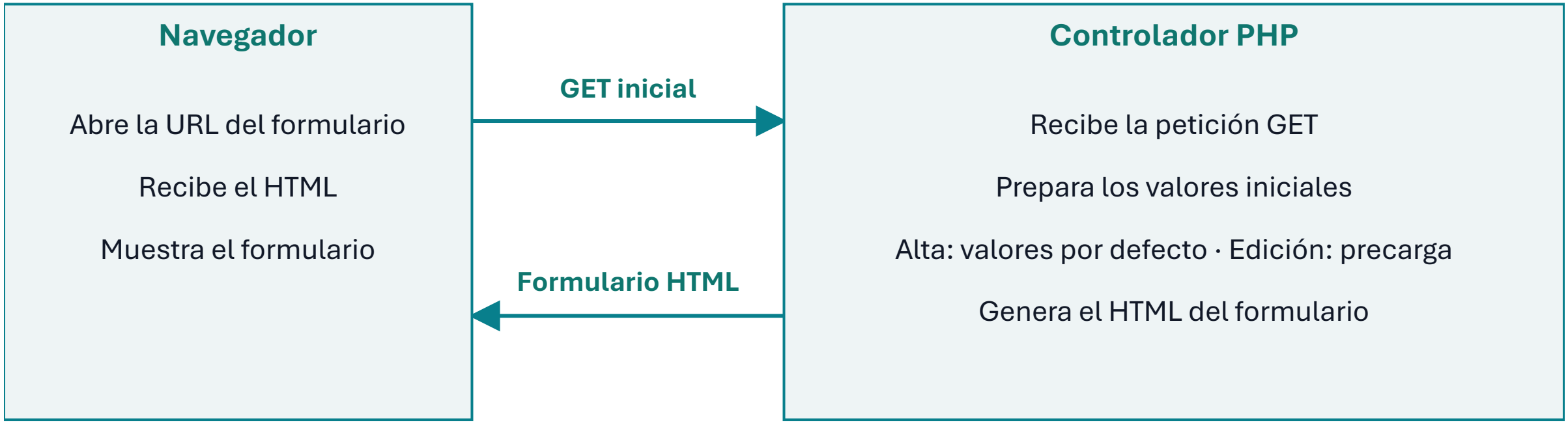
47–58

Integrar las operaciones

Repositorio, alta con PRG y edición.

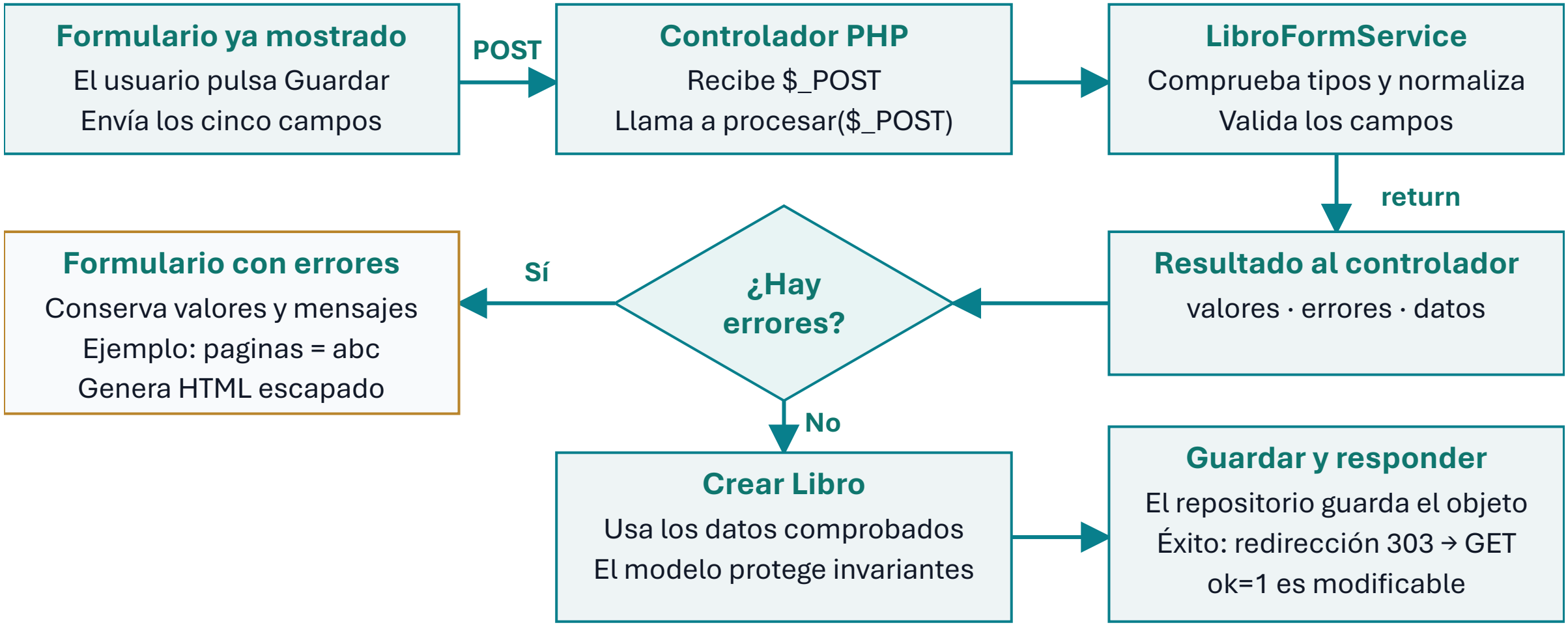
Flujo completo: GET inicial

RECIBIR DATOS



Flujo completo: envío POST

RECIBIR DATOS



Un formulario mínimo

RECIBIR DATOS

```
<form method="post" action="/libro-crear.php">
    <label for="titulo">Título</label>
    <input id="titulo" name="titulo" type="text">
    <button type="submit">Guardar</button>
</form>
```

method	action	name
Indica el método HTTP: aquí, POST.	Es la URL que recibe el envío.	El valor llegará con la clave titulo.

GET y POST

RECIBIR DATOS

Operación	Petición	Datos
Buscar	GET /libros.php?q=Dune	Los parámetros aparecen en la URL.
Dar de alta	POST /libro-crear.php	Los campos viajan en el cuerpo.
Recargar búsqueda	Nuevo GET	Repite una consulta.
Recargar tras alta	Puede reenviar POST	Podría repetir una operación.

POST no cifra los datos. HTTPS protege el transporte.

Dónde encuentra PHP la petición

RECIBIR DATOS

Variable	Qué contiene	Ejemplo
$_GET	Parámetros de la URL	q=Dune
$_POST	Campos de un formulario POST	titulo=Dune
$_SERVER	Información de la petición	REQUEST_METHOD=POST

Un POST puede tener además parámetros GET en su URL.

Leer un formulario POST

RECIBIR DATOS

```
$metodo = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$esPost = $metodo === 'POST';
$titulo = '';
if ($esPost) {
    $entrada = $_POST['titulo'] ?? '';
    if (!is_string($entrada)) {
        $entrada = '';
    }
    $titulo = trim($entrada);
}
```

Método

$esPost vale true en POST: entonces se ejecuta el if.

Entrada

Lee titulo del formulario anterior. Si falta o llega un array, usa una cadena vacía.

Resultado

Con « Dune », $titulo queda en «Dune». En GET permanece vacío.

Los controles envían valores

RECIBIR DATOS

HTML	Valor recibido	Comprobación del servidor
type="text"	Dune	Texto y longitud.
type="number"	"412", como cadena	Entero y rango.
type="email"	lector@example.org	Formato de correo.
type="date"	2026-10-24	Formato y fecha posible.

Las restricciones del navegador ayudan a introducir datos; PHP vuelve a comprobarlos.

Listas y texto de varias líneas

RECIBIR DATOS

```
<select name="genero">
    <option value="ciencia">Ciencia</option>
    <option value="fantasia">Fantasía</option>
</select>
<textarea name="observaciones">Primera edición</textarea>
```

Opción enviada	Texto	Servidor
Ciencia envía ciencia, el valor de option.	textarea envía su contenido.	La lista permitida se comprueba también en PHP.

Un checkbox puede no llegar

RECIBIR DATOS

HTML

```
<input type="checkbox"
       name="destacado" value="1">
```

Datos recibidos

```
// Marcado
['destacado' => '1']
// Desmarcado: no está la clave
[]
```

Comprobar la ausencia forma parte del tratamiento de este control.

id y name tienen funciones diferentes

RECIBIR DATOS

Sin name

```
<input id="titulo" type="text">
<!-- Escribes Dune -->
```

Con name

```
<input id="titulo" name="titulo"
       type="text">
<!-- $_POST['titulo']: Dune -->
```

id relaciona el control con label y permite localizarlo en el documento.

Búsqueda con GET

RECIBIR DATOS

```
<form method="get" action="/libros.php">
    <input name="q" type="text" value="Dune">
    <button type="submit">Buscar</button>
</form>

<!-- Petición resultante -->
<!-- GET /libros.php?q=Dune -->
```

Resultado

El filtro queda en la URL y puede compartirse.

PHP

Recoge q, comprueba su tipo y llama al servicio de consulta.

Alta con POST

RECIBIR DATOS

```
<form method="post" action="/libro-crear.php">
    <input name="titulo" value="Dune">
    <input name="autor" value="Frank Herbert">
    <input name="paginas" type="number" value="412">
    <button type="submit">Guardar</button>
</form>
```

Datos	Atención	Siguiente paso
titulo, autor y paginas llegan a $_POST.	paginas sigue llegando como cadena.	Comprobar la forma de los campos antes de utilizarlos.

Ausencia, cadena y array inesperado

VALIDAR Y RESPONDER

Lectura protegida

```
$entrada = $_POST['titulo'] ?? '';
if (!is_string($entrada)) {
    $errores['titulo'] =
        'Debe ser texto.';
    $entrada = '';
}
$titulo = trim($entrada);
```

Tres casos

```
// Falta titulo: ''
// titulo='  Dune  ': 'Dune'
// titulo[]=Dune:
// error y cadena vacía de trabajo
```

El error de tipo se conserva; no se transforma un array en un dato aceptado.

Tres operaciones sobre un mismo dato

VALIDAR Y RESPONDER

Operación	Ejemplo	Resultado
Normalizar	trim(" <Dune> ")	<Dune>
Validar	Comprobar que hay texto y no supera 120 caracteres	Se acepta como título.
Escapar al mostrar	htmlspecialchars con ENT_QUOTES y UTF-8	<Dune> en el HTML

El valor del dominio conserva el título. El escape se aplica al escribir HTML.

Obligatorio y longitud

VALIDAR Y RESPONDER

```
if (!isset($errores['titulo'])) {
    if ($titulo === '') {
        $errores['titulo'] = 'Escribe el título.';
    } elseif (mb_strlen($titulo, 'UTF-8') > 120) {
        $errores['titulo'] = 'Máximo 120 caracteres.';
    }
}
```

Antes	Casos	Límite
El título ya es una cadena recortada.	Dune se acepta; espacios solos fallan.	120 caracteres se aceptan; 121 se rechazan.

Un entero válido antes de convertir

VALIDAR Y RESPONDER

```
$paginas = is_string($entradaPaginas)
    ? filter_var($entradaPaginas,
        FILTER_VALIDATE_INT)
    : false;
if ($paginas === false || $paginas < 1
    || $paginas > 3000) {
    $errores['paginas'] = 'Entre 1 y 3000 páginas.';
}
```

Admitidos

"1", "412" y "3000".

Rechazados

"0", "3001", "12abc" y arrays.

UT2 aplicado

(int) "12abc" produce 12 y ocultaría el error.

Opciones permitidas y booleanos

VALIDAR Y RESPONDER

```
$permitidos = ['ciencia', 'fantasia', 'terror'];
if (!in_array($genero, $permitidos, true)) {
    $errores['genero'] = 'Elige un género válido.';
}
$marca = $_POST['destacado'] ?? null;
if ($marca !== null && $marca !== '1') {
    $errores['destacado'] = 'Valor no válido.';
}
$destacado = $marca === '1';
```

Lista

ciencia se acepta y aventura se rechaza.

Casilla

Ausente equivale a false; "1" equivale a true.

Manipulación

Un array o un valor distinto genera error.

filter_input y el dato original

VALIDAR Y RESPONDER

```
filter_input(int $type, string $var_name,
    int $filter = FILTER_DEFAULT, array|int $options = 0): mixed
```

```
$correo = filter_input(
    INPUT_POST, 'email', FILTER_VALIDATE_EMAIL
);
```

Parámetros

Origen INPUT_POST, nombre del campo y filtro explícito.

Retorno

Correo válido, false si no valida y null si falta.

Diferencia

Lee el dato original. filter_var valida un valor ya normalizado.

Correo sobre el valor normalizado

VALIDAR Y RESPONDER

```
$entrada = $_POST['email'] ?? '';
$email = is_string($entrada) ? trim($entrada) : '';
$correo = filter_var($email, FILTER_VALIDATE_EMAIL);

if (!is_string($entrada) || $correo === false) {
    $errores['email'] = 'Escribe un correo válido.';
}
```

Ejemplo de contacto	Rechazo	Normalización
" lector@example.org " se acepta tras recortar.	lector@ y un array no son correos válidos.	No se presenta convertir todo el correo a minúsculas como una obligación universal.

Crear una fecha opcional

VALIDAR Y RESPONDER

```
DateTimeImmutable::createFromFormat(string $format,
    string $datetime, ?DateTimeZone $timezone = null): DateTimeImmutable|false
```

```
$fecha = null;
if ($textoFecha !== '') {
    try {
        $fecha = DateTimeImmutable::createFromFormat(
            '!Y-m-d', $textoFecha
        );
    } catch (ValueError) {
        $fecha = false;
    }
}
```

Vacío

fechaAlta es opcional: una cadena vacía se representa con null.

Formato

!Y-m-d reinicia los campos y lee año, mes y día.

Antes

El valor ya es una cadena. Un byte NUL puede lanzar ValueError.

Comprobar una fecha no vacía

VALIDAR Y RESPONDER

```
if ($textoFecha !== '' && (
    $fecha === false
    || $fecha->format('Y-m-d') !== $textoFecha
)) {
    $errores['fechaAlta'] = 'Fecha no válida.';
}
```

Admitidos

Vacío → null. 2026-10-24 → objeto de fecha.

Rechazados

2026-02-31 y 24/10/2026 producen un error.

Motivo

Comparar con el texto enviado detecta los ajustes de fechas imposibles.

Resultado de validar el formulario

VALIDAR Y RESPONDER

Entrada

```
// Envío recibido
[
    'titulo' => '   ',
    'paginas' => '12abc',
    'genero' => 'ciencia',
]
```

Errores por campo

```
// Errores obtenidos
[
    'titulo' =>
        'Escribe el título.',
    'paginas' =>
        'Entre 1 y 3000 páginas.',
]
```

Los valores y los errores se conservan por separado. No se crea Libro si hay errores.

Conservar texto y mostrar el error

VALIDAR Y RESPONDER

```
<input name="titulo"
       value="<?= e($valores['titulo']) ?>">

<?php if (isset($errores['titulo'])): ?>
    <p><?= e($errores['titulo']) ?></p>
<?php endif; ?>
```

Valor	Escape	Resultado
Se vuelve a mostrar lo que has enviado.	e llama a htmlspecialchars con ENT_QUOTES y UTF-8, como en UT1.	Si falla páginas, el título no desaparece.

Conservar una selección y una casilla

VALIDAR Y RESPONDER

```
<option value="ciencia"
    <?= $valores['genero'] === 'ciencia'
        ? 'selected' : '' ?>>Ciencia</option>

<input type="checkbox" name="destacado" value="1"
    <?= $valores['destacado'] ? 'checked' : '' ?>>
```

Lista

selected recupera la opción enviada.

Checkbox

checked reproduce el booleano ya interpretado.

Seguridad

Los atributos de control salen de condiciones propias, no de HTML recibido.

Recorrido de un envío incorrecto

VALIDAR Y RESPONDER

Paso	Dato o resultado
1. Recibir	titulo="Dune", paginas="12abc"
2. Validar	El título es válido; páginas genera un error.
3. Decidir	No construir ni guardar el libro.
4. Responder	Formulario con Dune y el error de páginas.

La respuesta al POST incorrecto muestra el formulario en la misma petición.

Un recorrido completo: preparar

VALIDAR Y RESPONDER

```php
<?php
$esPost = ($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'POST';
$valores = ['titulo' => '', 'paginas' => ''];
$errores = [];
if ($esPost) {
    foreach ($valores as $campo => $inicial) {
        $v = $_POST[$campo] ?? '';
        if (!is_string($v)) {
            $errores[$campo] = 'Debe ser texto.';
            $v = '';
        }
        $valores[$campo] = trim($v);
    }
}
```

Un solo archivo

ejemplo.php se compone de estas cuatro diapositivas, en orden.

GET inicial

Valores vacíos y ningún error. Todavía no se valida el formulario.

POST

Se conservan dos cadenas normalizadas y los errores de tipo.

Un recorrido completo: validar

VALIDAR Y RESPONDER

```
if ($esPost) {
    if (!isset($errores['titulo'])
        && $valores['titulo'] === '') {
        $errores['titulo'] = 'Escribe el título.';
    }
    $n = filter_var($valores['paginas'],
        FILTER_VALIDATE_INT);
    if (!isset($errores['paginas'])
        && ($n === false || $n < 1 || $n > 3000)) {
        $errores['paginas'] = 'Entre 1 y 3000 páginas.';
    }
}
```

Dos reglas

En este ejemplo pequeño, título obligatorio y páginas entre 1 y 3000.

Sin ocultar errores

Si llegó un array, su error de tipo no se sustituye por otro mensaje.

Tipo comprobado

filter_var devuelve un entero o false; no se convierte antes de comprobar.

Un recorrido completo: decidir

VALIDAR Y RESPONDER

```
$datos = null;
if ($esPost && $errores === []) {
    $datos = [
        'titulo' => $valores['titulo'],
        'paginas' => $n,
    ];
}
```

POST válido

" Dune " y "412": valores guarda "Dune" y "412"; datos contiene el entero 412.

POST incorrecto

"Dune" y "12abc": datos es null; errores contiene páginas y valores conserva "12abc".

GET inicial

datos sigue en null. Mostrar el formulario no implica procesar una operación.

Un recorrido completo: responder

VALIDAR Y RESPONDER

```
function e(string $texto): string {
    return htmlspecialchars($texto,
        ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
?>
<form method="post">
    <input name="titulo" value="<?= e($valores['titulo']) ?>">
    <p><?= e($errores['titulo'] ?? '') ?></p>
    <input name="paginas" value="<?= e($valores['paginas']) ?>">
    <p><?= e($errores['paginas'] ?? '') ?></p>
    <button type="submit">Comprobar</button>
</form>
```

Tras un error se conservan Dune y 12abc, junto al mensaje de páginas. La función e escapa el texto al generar HTML.

Responsabilidades del mismo ejemplo

ORGANIZAR LA VALIDACIÓN

Archivo o clase	Trabajo concreto
public/libro-crear.php	Recibe la petición y decide la respuesta.
LibroFormService	Prepara valores, valida y devuelve errores.
Libro	Conserva el estado y protege sus invariantes.
LibroRepository	Obtiene y guarda objetos mediante un contrato.

Estas responsabilidades recuperan la organización estudiada en UT2.

Los cinco campos de la ficha

ORGANIZAR LA VALIDACIÓN

Campo	Valor del formulario	Dato válido para Libro
titulo	Cadena recortada, obligatoria	Texto de hasta 120 caracteres.
autor	Cadena recortada, obligatoria	Texto no vacío.
paginas	Cadena, por ejemplo "412"	Entero entre 1 y 3000.
genero	Cadena de la selección	ciencia, fantasia o terror.
fechaAlta	Cadena Y-m-d o vacía	DateTimeImmutable o null.

Correo, observaciones y destacado ilustran otros controles; no forman parte de esta ficha de Libro.

Una regla reutilizable en Validator


```
final class Validator
{
    public function requerido(
        string $valor, string $mensaje
    ): ?string {
        return trim($valor) === '' ? $mensaje :
            null;
    }
}
```

Contrato

Recibe el texto y el mensaje que se mostrará si falla.

Retorno

null significa que cumple la regla; string contiene el error.

Extracción

La regla ya utilizada se reúne en una clase para reutilizarla.

Usar la regla sin duplicarla

Llamadas

```
$v = new Validator();
$ok = $v->requerido('Dune',
    'Escribe el título.');
$error = $v->requerido('   ',
    'Escribe el título.');
```

Resultados

```
$ok    // null
$error // 'Escribe el título.'
if ($error !== null) {
    $errores['titulo'] = $error;
}
```

Se comprueba !== null para guardar solo los errores.

El contrato de LibroFormService

ORGANIZAR LA VALIDACIÓN

```
public function __construct(private Validator $v) {}

public function procesar(array $entrada): array
{
    // El desarrollo completo aparece a continuación.
    return [
        'valores' => $valores,
        'errores' => $errores,
        'datos' => $datos,
    ];
}
```

Validator

El constructor recibe la regla reutilizable y la conserva en $this->v.

Resultado

valores redibuja los controles; errores asocia mensajes a campos; datos contiene tipos comprobados.

Esquema

Este bloque presenta el contrato. Las cinco partes siguientes desarrollan procesar sin omitir instrucciones.

Una regla completa del servicio

ORGANIZAR LA VALIDACIÓN

```
public function validarPaginas(mixed $entrada):
    ?string
{
    $n = is_string($entrada)
        ? filter_var($entrada, FILTER_VALIDATE_INT)
        : false;
    if ($n === false || $n < 1 || $n > 3000) {
        return 'Entre 1 y 3000 páginas.';
    }
    return null;
}
```

Parte ejecutable

validarPaginas pertenece a LibroFormService. procesar la llama para el campo páginas.

Uso

validarPaginas("412") devuelve null. validarPaginas("12abc") devuelve el mensaje.

Contrato anterior

Las cinco partes siguientes completan procesar y utilizan esta regla sin copiarla.

Servicio: preparar los valores

ORGANIZAR LA VALIDACIÓN

```
public function procesar(array $entrada): array
{
    $valores = [];
    $errores = [];
    $campos = ['titulo', 'autor', 'paginas',
        'genero', 'fechaAlta'];
    foreach ($campos as $campo) {
        $v = $entrada[$campo] ?? '';
        if (!is_string($v)) {
            $errores[$campo] = 'Debe ser texto.';
            $v = '';
        }
        $valores[$campo] = trim($v);
    }
```

procesar · 1 de 5

Comienza el método de LibroFormService. Las cinco partes se escriben consecutivamente.

Entrada

Solo se leen los cinco campos de la ficha; una clave ausente toma una cadena vacía.

Salida de esta parte

valores contiene cadenas; errores conserva cualquier tipo inesperado.

Servicio: reutilizar Validator

ORGANIZAR LA VALIDACIÓN

```
$mensajes = ['titulo' => 'Escribe el título.',
        'autor' => 'Escribe el autor.'];
    foreach ($mensajes as $campo => $mensaje) {
        if (isset($errores[$campo])) {
            continue;
        }
        $error = $this->v->requerido(
            $valores[$campo], $mensaje);
        if ($error !== null) {
            $errores[$campo] = $error;
        }
    }
    if (!isset($errores['titulo'])
        && mb_strlen($valores['titulo'], 'UTF-8') > 120) {
        $errores['titulo'] = 'Máximo 120 caracteres.';
    }
```

procesar · 2 de 5

Se aplica requerido al título y al autor mediante el Validator recibido.

continue

Un error de tipo ya registrado no se reemplaza por un error de campo obligatorio.

Longitud

Después de comprobar que existe el título, se comprueba su límite de 120 caracteres.

Servicio: páginas y género

ORGANIZAR LA VALIDACIÓN

```
if (!isset($errores['paginas'])) {
        $error = $this->validarPaginas($valores['paginas']);
        if ($error !== null) {
            $errores['paginas'] = $error;
        }
    }
    $permitidos = ['ciencia', 'fantasia', 'terror'];
    $genero = $valores['genero'];
    if (!isset($errores['genero'])
        && !in_array($genero, $permitidos, true)) {
        $errores['genero'] = 'Elige un género válido.';
    }
```

procesar · 3 de 5

validarPaginas es el método completo mostrado antes; devuelve un mensaje o null.

Sin conversión prematura

"12abc" conserva su texto y produce un error. Todavía no se convierte a entero.

Lista permitida

El servidor acepta únicamente las tres opciones de la ficha.

Servicio: fecha opcional

ORGANIZAR LA VALIDACIÓN

```
$fecha = null;
    $textoFecha = $valores['fechaAlta'];
    if (!isset($errores['fechaAlta'])
        && $textoFecha !== '') {
        try {
            $fecha = DateTimeImmutable::createFromFormat(
                '!Y-m-d', $textoFecha
            );
        } catch (ValueError) {
            $fecha = false;
        }
        if ($fecha === false
            || $fecha->format('Y-m-d') !== $textoFecha) {
            $errores['fechaAlta'] = 'Fecha no válida.';
        }
    }
```

procesar · 4 de 5

Se reutiliza el tratamiento de fechas ya explicado.

Vacío permitido

Una fecha vacía permanece como null. Una fecha imposible genera un error.

Tipo inesperado

Si llegó un array, su error permanece aunque la cadena de trabajo sea vacía.

Servicio: devolver el resultado

ORGANIZAR LA VALIDACIÓN

```
$datos = null;
    if ($errores === []) {
        $datos = [
            'titulo' => $valores['titulo'],
            'autor' => $valores['autor'],
            'paginas' => (int) $valores['paginas'],
            'genero' => $valores['genero'],
            'fechaAlta' => $fecha,
        ];
    }
    return [
        'valores' => $valores, 'errores' => $errores,
        'datos' => $datos,
    ];
}
```

procesar · 5 de 5

Termina el método: su resultado siempre tiene las mismas tres claves.

Conversión segura

Se convierte páginas a entero después de validar el texto completo y su rango.

Con errores

datos es null; valores y errores permiten responder sin crear ni modificar Libro.

El servicio con un envío válido

ORGANIZAR LA VALIDACIÓN

Entrada

```
$formulario = new LibroFormService(
    new Validator()
);
$r = $formulario->procesar([
    'titulo' => '  Dune  ',
    'autor' => 'Frank Herbert',
    'paginas' => '412',
    'genero' => 'ciencia',
    'fechaAlta' => '',
]);
```

Resultado observado

```
$r['valores']['paginas']; // '412'
$r['errores'];            // []
$r['datos'];
// [
//   'titulo' => 'Dune',
//   'autor' => 'Frank Herbert',
//   'paginas' => 412,
//   'genero' => 'ciencia',
//   'fechaAlta' => null,
// ]
```

valores conserva cadenas para los controles; datos entrega páginas como entero y la fecha opcional como null.

El servicio con un envío incorrecto

Entrada

```
$r = $formulario->procesar([
    'titulo' => ['Dune'],
    'autor' => 'Frank Herbert',
    'paginas' => '12abc',
    'genero' => 'ciencia',
    'fechaAlta' => '',
]);
```

Resultado observado

```
$r['valores']['titulo'];  // ''
$r['valores']['paginas']; // '12abc'
$r['errores'];
// [
//   'titulo' => 'Debe ser texto.',
//   'paginas' =>
//       'Entre 1 y 3000 páginas.',
// ]
$r['datos']; // null
```

El formulario vuelve a mostrar 12abc y sus errores; no se construye Libro.

Formulario e invariantes del objeto

ORGANIZAR LA VALIDACIÓN

Responsable	Ejemplo	Resultado
Formulario	Recibe paginas="abc"	Devuelve un mensaje junto al campo.
Formulario	Recibe un array en titulo	Lo rechaza antes de usar trim.
Libro	Se construye con título vacío	Lanza InvalidArgumentException.
Libro	Se construye con páginas negativas	Impide crear un estado inválido.

El constructor sigue protegiendo las reglas aunque no se utilice un formulario.

Un único contrato de Libro

ORGANIZAR LA VALIDACIÓN

```
public function __construct(
    private string $titulo,
    private string $autor,
    private int $paginas,
    private string $genero = 'ciencia',
    private ?DateTimeImmutable $fechaAlta = null,
) { /* protege las invariantes */ }

$libro = new Libro('Dune', 'Frank Herbert', 412);
```

La misma API

Formularios y tests utilizan este constructor.

Estado

titulo(), autor(), paginas(), genero() y fechaAlta() consultan los datos.

Estado conservado

La edición de la ficha conserva la disponibilidad y la portada del objeto.

Repositorio e identificadores

INTEGRAR LAS OPERACIONES

```
interface LibroRepository
{
    public function buscarPorId(int $id): ?Libro;
    public function guardar(Libro $libro): int;
    public function actualizar(int $id, Libro
        $libro): void;
}
// ArrayLibroRepository usa IDs como claves del
// array.
// guardar devuelve un ID nuevo; actualizar conserva
// el ID.
```

Ejemplo

guardar($libro) devuelve 1; buscarPorId(1) devuelve ese objeto.

Duración

El repositorio se reconstruye en cada petición. Un alta no sobrevive a la redirección.

Alta: preparar y procesar el envío

INTEGRAR LAS OPERACIONES

```
$esPost = ($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'POST';
$formulario = new LibroFormService(new Validator());
$valores = ['titulo' => '', 'autor' => '', 'paginas' => '',
    'genero' => 'ciencia', 'fechaAlta' => ''];
$errores = [];
$operacionCorrecta = false;
if ($esPost) {
    $r = $formulario->procesar($_POST);
    $valores = $r['valores'];
    $errores = $r['errores'];
}
```

GET inicial

El formulario utiliza los cinco valores iniciales y no ejecuta procesar.

POST

Se recuperan valores y errores del servicio ya desarrollado.

Instancias

new crea Validator y el servicio; el autoloader de UT2 permite cargar sus clases.

Alta: guardar y marcar el éxito

INTEGRAR LAS OPERACIONES

```
if ($esPost && $errores === []) {
    $d = $r['datos'];
    $libro = new Libro(
        $d['titulo'], $d['autor'], $d['paginas'],
        $d['genero'], $d['fechaAlta']
    );
    $id = $repo->guardar($libro);
    $operacionCorrecta = true;
}
```

Antes

$repo es el repositorio simulado de UT2. Este bloque sigue al procesamiento del formulario.

Decisión

Solo un POST sin errores construye Libro y ejecuta guardar.

Después

El éxito se marca tras guardar. El siguiente bloque podrá devolver la redirección.

Recargar después de un POST correcto

Acción	Sin redirección	Con PRG
Enviar	POST procesa el alta	POST procesa el alta
Respuesta	HTML de confirmación	303 con una URL de destino
Navegación final	La petición actual sigue siendo POST	El navegador realiza un GET
Recargar	Puede reenviar los datos	Repite el GET final

PRG evita el reenvío por recarga; no sustituye al almacenamiento ni garantiza por sí solo que no haya duplicados.

Redirección tras la operación


```
header(string $header, bool $replace = true,
    int $response_code = 0): void
```

```
// Al inicio: $operacionCorrecta = false;
// Solo cambia a true al completar guardar o
// actualizar.
if ($esPost && $operacionCorrecta) {
    header('Location: /resultado.php?ok=1', true,
        303);
    exit;
}
```

Condición

Un GET inicial no entra en esta rama. Tampoco un POST fallido.

Orden

Enviar la redirección antes de cualquier salida.

303 y exit

303 conduce a un GET. exit termina el script.

Edición: validar el ID y buscar

INTEGRAR LAS OPERACIONES

```
$entradaId = $_GET['id'] ?? '';
$id = is_string($entradaId)
    ? filter_var($entradaId, FILTER_VALIDATE_INT)
    : false;
$libro = ($id !== false && $id > 0)
    ? $repo->buscarPorId($id) : null;
if ($libro === null) {
    $erroresGenerales[] =
        'No se encuentra el libro.';
}
```

Casos

ID 1 existente, ID 999 ausente, abc inválido e id[]=1 inesperado.

Método

La URL identifica el libro tanto en GET como en POST.

Edición: precargar los cinco campos

INTEGRAR LAS OPERACIONES

```
$esPost = ($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'POST';
$errores = [];
$operacionCorrecta = false;
$valores = [
    'titulo' => $libro->titulo(),
    'autor' => $libro->autor(),
    'paginas' => (string) $libro->paginas(),
    'genero' => $libro->genero(),
    'fechaAlta' => $libro->fechaAlta()
        ?->format('Y-m-d') ?? '',
];
```

Condición

Se ejecuta solo cuando el libro existe. El repositorio parte de los objetos de ejemplo de UT2.

Representación

Páginas vuelve a ser texto; la fecha vuelve a Y-m-d. null se representa como cadena vacía.

Estado

Género y fecha también se precargan: no se pierden al editar otro campo.

Edición: conservar el envío si falla

INTEGRAR LAS OPERACIONES

```
if ($esPost) {
    $r = $formulario->procesar($_POST);
    $valores = $r['valores'];
    $errores = $r['errores'];
}
```

Antes

Se crea LibroFormService con un Validator, igual que en el alta.

GET inicial

Se mantienen los cinco valores precargados del objeto.

POST incorrecto

Los valores enviados sustituyen a la precarga. Así se conservan las correcciones del usuario.

El mismo formulario de cinco campos


```
<form method="post">
    <input name="titulo" value="<?= e($valores['titulo']) ?>">
    <input name="autor" value="<?= e($valores['autor']) ?>">
    <input name="paginas" value="<?= e($valores['paginas']) ?>">
    <!-- A continuación: género, fecha y botón -->
```

Situación	Origen de valores	Resultado visible
Alta inicial	Valores vacíos y género inicial	Ficha por completar.
Edición inicial	Los cinco campos de Libro	Ficha existente, incluida su fecha.
POST incorrecto	valores devuelto por el servicio	Texto enviado y errores por campo.

Representar el género y la fecha


```
<select name="genero">
    <?php foreach (['ciencia', 'fantasia', 'terror'] as $g): ?>
        <option value="<?= e($g) ?>"
            <?= $valores['genero'] === $g ? 'selected' : '' ?>>
            <?= e($g) ?>
        </option>
    <?php endforeach; ?>
    </select>
    <input name="fechaAlta" type="date"
        value="<?= e($valores['fechaAlta']) ?>">
    <button type="submit">Guardar</button>
</form>
```

Se utiliza e y se muestran los errores junto a cada campo como en el ejemplo completo. Sin action, la edición mantiene la URL con su id.

Actualizar los datos editables

INTEGRAR LAS OPERACIONES

```
public function actualizarDatos(
    string $titulo, string $autor, int $paginas,
    string $genero, ?DateTimeImmutable $fechaAlta
): void {
    $this->validarFicha($titulo, $autor, $paginas,
        $genero);
    $this->titulo = trim($titulo);
    $this->autor = trim($autor);
    $this->paginas = $paginas;
    $this->genero = $genero;
    $this->fechaAlta = $fechaAlta;
}
```

Invariante

validarFicha aplica las mismas reglas internas que el constructor y lanza excepción antes de asignar.

Conservación

No modifica la disponibilidad ni la referencia de la portada.

Edición sin perder el estado anterior

INTEGRAR LAS OPERACIONES

```
if ($esPost && $libro !== null && $errores === []) {
    try {
        $d = $r['datos'];
        $libro->actualizarDatos(
            $d['titulo'], $d['autor'],
                $d['paginas'],
            $d['genero'], $d['fechaAlta']
        );
        $repo->actualizar($id, $libro);
        $operacionCorrecta = true;
    } catch (OutOfBoundsException) {
        $erroresGenerales[] =
            'El libro ya no existe.';
    }
}
```

Antes

El libro existe. Los datos del formulario ya son válidos.

Estado

actualizarDatos modifica la ficha y conserva disponibilidad y portada.

Resultado

El ID se conserva. Tras actualizar, el mismo bloque PRG del alta redirige antes de generar HTML.